Punto de Venta Farmacia

Cómo explicar lo que hicimos en Firebase: arquitectura, reglas de seguridad y concurrencia

---

1. Qué es y qué usamos

Es una app web (HTML + JavaScript) sin servidor propio. Usa tres servicios de Firebase:

- **Authentication:** el inicio de sesión con usuario y contraseña.
- **Firestore:** la base de datos compartida, en la nube y en tiempo real.
- **Hosting:** publica la página en una URL (farmacia-pos-valde.web.app).

Todos los usuarios leen y escriben en la **misma base de datos**. Como no hay un servidor intermedio, las **reglas de seguridad** son lo que decide quién puede hacer qué.

2. Datos (colecciones de Firestore)

Colección	Contenido
users	Un documento por usuario con nombre, usuario y **rol** (cliente o admin).
products	Catálogo con precio, existencia, caducidad y si requiere receta.
sales	Cada venta, con artículos, total, método de pago y quién la hizo.
counters	El último folio usado, para que los tickets sean consecutivos.

3. Roles y cómo se aplican

El rol vive en `users/{uid}.role`. Se aplica en dos capas:

- **La interfaz** muestra u oculta pantallas según el rol. Esto es solo comodidad.
- **Las reglas** (`firestore.rules`) son la protección real. Aunque alguien manipule la página desde la consola del navegador, el servidor rechaza lo que no le corresponde.

Idea clave para decirle al profesor: **ocultar un botón no es seguridad; las reglas sí.**

4. Las reglas, colección por colección

La función base es `isAdmin()`: busca el documento del usuario en `users` y revisa que su rol sea admin.

Colección	Quién puede qué
users	Cada quien lee solo su perfil; el admin lee todos. Al registrarse, un usuario solo puede crear **su** perfil y **solo como cliente**: nadie se auto-asigna admin. Solo el admin cambia roles o crea cuentas.
products	Todos los usuarios con sesión leen. Solo el admin crea, edita o borra. Un cliente solo puede **bajar** el campo stock (nunca subirlo ni tocar precio u otros campos). Eso permite comprar sin darle permisos de edición.
sales	El admin ve todas; el cliente solo las suyas (userId igual a su uid). Las ventas **no se pueden editar ni borrar**, para que el historial no se altere.
counters	El folio solo puede avanzar de uno en uno (ver sección 8).

5. Concurrencia (lo que pidió el profesor)

- **Sin sobreventa:** la compra usa `runTransaction`. En una sola operación atómica lee el stock actual, valida, descuenta, genera el folio y guarda la venta. Si dos personas compran la última pieza a la vez, Firestore reintenta una de las transacciones con datos frescos, ve que ya no hay y la rechaza con un aviso.
- **Tiempo real:** `onSnapshot` mantiene todas las pantallas sincronizadas. Si alguien compra, el stock cambia solo en las demás y el carrito se ajusta.
- **Ediciones del admin:** el stock se ajusta con `increment(delta)`, así que corregir un producto mientras se vende no pisa las ventas que llegaron en medio.

6. Cómo demostrarlo

Abran dos ventanas (una en incógnito) con dos cuentas. Dejen un producto con 1 pieza, pónganlo en ambos carritos y paguen casi al mismo tiempo. Una compra pasa y la otra dice que se agotó. Esa demo vale más que cualquier explicación.

7. Limitaciones (conviene decirlas antes de que pregunten)

- Las reglas no pueden verificar que el **total** de una venta cuadre con los precios. Un usuario técnico podría mandar un total falso. La solución profesional es hacer el cobro en un servidor (Cloud Functions), que requiere el plan de pago.
- Un cliente podría bajar stock sin generar venta, porque la regla solo comprueba que baje (ver sección 9).

- Firebase Auth pide correo, así que internamente usamos `usuario@farmacia.app`. No hay recuperación de contraseña.

Decirlo con honestidad suma puntos: muestra que entienden el modelo de seguridad y no solo que funciona.

8. Qué es la colección counters

Es el contador de **folios**: que los tickets salgan #0001, #0002, #0003, sin repetirse ni saltarse.

Firestore no tiene autoincremento como SQL Server, así que lo hacemos a mano con un documento, `counters/sales`, que guarda solo el último número usado:

```
counters/sales  ->  { last: 7 }
```

En cada compra, dentro de la misma transacción:

- Lee `last` (7).
- El folio de esta venta es 8.
- Guarda `last: 8` y crea la venta con `folio: 8`.

Como todo va en la transacción, si dos personas compran al mismo tiempo no pueden recibir el mismo folio. Una lee 7 y escribe 8; la otra, que también leyó 7, es reintentada por Firestore, lee 8 y se queda con el 9.

Las reglas protegen el contador de dos maneras:

- Solo se puede **crear** si `last == 1`, es decir, la primera venta de todas.
- Solo se puede **actualizar** si el nuevo valor es `last + 1`.

Así nadie puede regresarlo a 0, saltarlo a 9999 ni repetir un número, ni siquiera desde la consola del navegador.

```
allow create: if signedIn() && request.resource.data.last == 1;
allow update: if signedIn() && request.resource.data.last == resource.data.last + 1;
```

9. Por qué un cliente podría bajar el stock sin comprar

La regla de `products` dice, para un cliente: "puedes modificar un producto solo si cambias únicamente stock y updatedAt, y el nuevo stock es menor que el actual". Es lo mínimo necesario para que comprar funcione, porque comprar implica descontar.

```
allow update: if isAdmin() || (
  request.resource.data.diff(resource.data).affectedKeys().hasOnly(['stock','updatedAt'])
  && request.resource.data.stock >= 0
  && request.resource.data.stock < resource.data.stock
);
```

El problema es que **cada regla evalúa una escritura a la vez**. No sabe si esa escritura viene acompañada de una venta. La app hace tres cosas juntas en la transacción:

```
1. baja el stock del producto
2. avanza el contador
3. crea la venta
```

Pero alguien con conocimientos técnicos podría abrir la consola del navegador y mandar solo el paso 1:

```
updateDoc(doc(db, 'products', 'idDelParacetamol'), { stock: 0 })
```

Esa escritura cumple la regla (solo cambia stock y baja), así que Firestore la acepta. El producto aparecería agotado sin que exista ninguna venta.

Aspecto	Detalle
Qué daño hace	Es sabotaje, no robo. No se lleva dinero ni gana nada; solo deja el inventario mal. El admin lo corrige editando el producto.
Por qué no se arregla en las reglas	Las reglas no pueden recorrer una lista de productos para comprobar que hay una venta por cada descuento (no tienen ciclos). Hay mitigaciones parciales, pero ninguna cierra el hueco por completo.
Cómo se arregla de verdad	Que el cliente **no tenga permiso de escribir** en products. La compra la haría una **Cloud Function** (un programa en el servidor de Google) que valida todo y hace el descuento con permisos de administrador. Costo: Cloud Functions exige el plan de pago de Firebase (Blaze).

10. Cómo decírselo al profesor

> *"Con el plan gratuito no hay servidor propio, así que la lógica de compra corre en el cliente y se protege con reglas más una transacción. Eso garantiza que no haya sobreventa ni folios repetidos. Reconocemos que un usuario malicioso podría descontar stock sin generar una venta, y que la solución profesional es mover el cobro a Cloud Functions."*

11. Preguntas que podría hacer

¿Qué pasa si alguien edita el JavaScript?

Las reglas se ejecutan en el servidor de Firebase, no en el navegador, así que no se pueden saltar.

¿Cómo se vuelve alguien admin?

Solo otro admin, o editando la base desde la consola del proyecto, que solo ustedes controlan.

¿Por qué una transacción y no un simple update?

Con un update, dos lecturas simultáneas del mismo stock podrían descontar ambas y vender de más (condición de carrera). La transacción reintenta con datos frescos y lo evita.

12. Validación de transacciones por red

El profesor preguntó si validamos las transacciones sobre la red. Una transacción es atómica en el servidor, pero la **red** puede fallar en dos momentos distintos:

- **Antes de confirmar:** no queda ningún cambio (todo o nada). El usuario ve el error y puede reintentar.
- **Después de confirmar, antes de que llegue la respuesta:** la venta sí se guardó, pero el usuario cree que falló. Si vuelve a pagar, se duplicaría la venta y el descuento de stock. Este era el hueco que teníamos.

Cómo lo resolvimos

- **Compra idempotente:** cada intento de pago genera un `saleId` en el cliente. La transacción primero busca esa venta: si ya existe, la devuelve sin descontar stock ni gastar otro folio. El reintento usa el mismo id.
- **Aviso sin conexión:** la app detecta que no hay internet, muestra una barra roja y bloquea el botón de pagar. Si la red falla en pleno pago, avisa que el pedido sigue en el carrito y que no se cobrará doble.
- **Reintento por conflicto:** si otra compra gana la carrera, la transacción se repite con datos frescos y responde "se agotó" en lugar de un error técnico.

Pruebas automatizadas (carpeta tests/)

Corren en el emulador de Firestore con las reglas reales y el mismo código de compra de la app. Para simular la caída de red se coloca un proxy TCP entre el cliente y el emulador que corta las conexiones.

#	Prueba	Resultado esperado
1	Compra normal	Crea la venta, descuenta stock y avanza el folio
2	Respuesta perdida (reintento tras confirmar)	No se duplica la venta; el stock se descuenta una sola vez
3	Doble clic (dos envíos simultáneos)	Una sola venta
4	Red caída	Falla con error de red, no cambia nada; al volver la red, el reintento funciona
5	Atomicidad con varios productos	Si uno no alcanza, no se descuenta ninguno
6	4 clientes, 1 pieza	Gana uno; los demás reciben "se agotó"
7	Folios con compras simultáneas	Folios distintos y consecutivos
8	Seguridad	Un cliente no sube stock ni lee ventas ajenas

Resultado: 8 de 8 pruebas correctas. Para comprobar que las pruebas son válidas, desactivamos a propósito la protección contra duplicados y las pruebas 2 y 3 fallaron, así que sí detectan el problema. Se ejecutan con `cd tests && npm install && npm test`.

Límite: las pruebas simulan la red en una sola computadora. Falta repetir la prueba de concurrencia con varias personas en sus propios equipos sobre la URL publicada, y anotar el resultado.